04 - 优化与部署面试题

> 共 22 道题，涵盖 KV-Cache、MoE、YaRN、生成策略、FlashAttention、量化、推理优化、Tool Calling 等。

Q1: KV-Cache 的原理是什么？为什么能加速推理？

详细版：

自回归生成时，模型每一步生成一个 token，每个新 token 都需要与之前所有 token 做注意力计算。如果每步都重新计算所有 token 的 K 和 V，大量计算是重复的。

KV-Cache 的做法：把已经计算过的 K 和 V 缓存起来，每步只计算新 token 的 Q、K、V，然后将新的 K、V 追加到缓存中：

```
# 第 t 步（生成第 t 个 token）
q_new = linear_q(x_t)              # 只计算新 token 的 Q
k_new = linear_k(x_t)              # 只计算新 token 的 K
v_new = linear_v(x_t)              # 只计算新 token 的 V

k_cache = torch.cat([k_cache, k_new], dim=seq_len)  # 追加到缓存
v_cache = torch.cat([v_cache, v_new], dim=seq_len)

attn = softmax(q_new @ k_cache.T / sqrt(d_k)) @ v_cache  # 新 Q 与全部 KV 做注意力
```

加速原理： - 不使用 KV-Cache：每步的计算量 $O(n^2d)$（n 是已生成长度） - 使用 KV-Cache：每步的计算量 $O(nd)$（只有新 Q 与所有 K 做点积） - 省掉了重复计算之前 token 的 K 和 V 的线性投影

代价：需要额外显存来存储 KV-Cache。

精简版：

KV-Cache 缓存已计算的 K/V，每步只算新 token 的 Q/K/V。避免重复计算，将每步复杂度从 $O(n^2d)$ 降到 $O(nd)$。代价是需要额外显存。

Q2: 为什么训练时不用 KV-Cache?

详细版：

训练时不需要 KV-Cache 的根本原因是 **Teacher Forcing**：

1. **训练是并行的**：通过 Causal Mask，所有位置的 Q、K、V 同时计算，一次矩阵乘法完成整个序列的注意力。不存在"逐 token 生成"的过程。

2. **KV-Cache 是给自回归逐步生成用的**：推理时一个一个生成 token，才有"之前的 KV 不变"这个条件。训练时输入序列是完整已知的。

3. **训练需要所有位置的梯度**：每个位置都参与 loss 计算和梯度回传。KV-Cache 的意义在于避免重复前向计算，但训练的前向计算本来就是并行一次性完成的。

类比：训练就像"开卷考试"（答案全部已知，一起处理），推理就像"现场作文"（一个字一个字写，需要记住前面写了什么）。

精简版：

训练用 Teacher Forcing 并行计算所有位置，不存在逐 token 生成，不需要缓存。KV-Cache 专为推理时的自回归逐步生成设计。

Q3: KV-Cache 的内存占用怎么计算?

详细版：

KV-Cache 存储所有层的 K 和 V 张量。

单层单头的 KV-Cache 大小： - K: (batch_size, seq_len, d_head) → batch_size × seq_len × d_head 个元素 - V: (batch_size, seq_len, d_head) → 同上

总 KV-Cache 大小（FP16）：

```
memory = 2 × n_layers × kv_heads × seq_len × d_head × batch_size × 2 bytes
```

（2 表示 K 和 V，最后的 2 bytes 是 FP16）

MiniMind 的 KV-Cache（batch_size=1, seq_len=512）：

```
= 2 × 8 × 4 × 512 × 96 × 1 × 2 bytes
= 2 × 8 × 4 × 512 × 96 × 2
= 6,291,456 bytes ≈ 6 MB
```

GQA 对 KV-Cache 的影响： - MHA（8 个 KV 头）的 KV-Cache 是 GQA（4 个 KV 头）的 2 倍 - 这就是 GQA 的核心价值：减少 KV-Cache 的显存占用

对于大模型（如 LLaMA-70B, 80 层, 8 KV 头, d_head=128），一个 4K 序列的 KV-Cache 就要约 5 GB。

精简版：

KV-Cache = 2 × n_layers × kv_heads × seq_len × d_head × 2bytes(FP16)。MiniMind(8层, 4KV头,d=96) 在 512 长度下约 6MB。GQA 的核心价值就是减半 KV-Cache。

Q4: MoE（Mixture of Experts）的原理是什么?

详细版：

MoE 的核心思想：将 FFN 层替换为多个"专家"（Expert），每个 token 只路由到部分专家计算，从而在增加模型容量的同时不增加（或少量增加）计算量。

结构：

```
MoE-FFN(x) = Σᵢ g_i(x) · Expert_i(x)
```

其中 g(x) 是路由函数（Router/Gate），决定每个 token 分配给哪些专家。

Router（通常是一个线性层 + softmax）：

```
g(x) = softmax(W_gate · x)  →  选 Top-K 个专家
```

典型配置： - 8 或 16 个专家 - Top-K = 2（每个 token 路由到 2 个专家） - 每个 Expert 结构与标准 FFN 相同

优势： - 参数量大幅增加（8 个专家 → 参数量约 8 倍），但激活参数不变（只用 Top-K 个） - 模型容量更大，但推理速度（FLOPs）基本不变

精简版：

MoE 将 FFN 替换为多个专家，每个 token 通过 Router 只路由到 Top-K 个专家。增加模型容量（参数量）但不增加计算量（每个 token 只激活 K 个专家）。

Q5: MoE 的 Router 是怎么工作的？Top-K 怎么选？

详细版：

Router 计算：

```
router_logits = self.gate(x)            # (batch, seq, n_experts)
router_probs = F.softmax(router_logits, dim=-1)
top_k_probs, top_k_indices = torch.topk(router_probs, k=K, dim=-1)
top_k_probs = top_k_probs / top_k_probs.sum(dim=-1, keepdim=True)  # 归一化
```

Top-K 的选择： - **K=1**：每个 token 只去一个专家。计算最快，但可能损失质量 - **K=2**：主流选择（Mixtral、DeepSeek-MoE）。平衡了质量和效率 - **K>2**：增加计算成本，收益递减

加权方式：Top-K 选出的专家按归一化后的概率加权求和：

```
output = Σᵢ∈TopK  (prob_i / Σ prob) · Expert_i(x)
```

精简版：

Router 是一个线性层 + softmax，为每个 token 计算每个专家的概率。选 Top-K（通常 K=2）个概率最高的专家，按归一化概率加权求和。

Q6: MoE 中的负载均衡 Loss 是什么？为什么需要？

详细版：

问题：专家坍缩（Expert Collapse） - 训练初期，某些专家偶然获得更多 token - 这些专家训练得更好 → 获得更高的路由概率 → 获得更多 token（正反馈） - 最终大部分 token 都路由到少数几个专家，其余专家闲置（"死专家"） - 模型退化为普通 FFN，MoE 的容量优势消失

负载均衡 Loss（Auxiliary Load Balancing Loss）：

```
L_aux = α · N · Σᵢ f_i · P_i
```

其中： - N：专家数量 - f_i：第 i 个专家实际处理的 token 比例 - P_i：Router 分配给第 i 个专家的平均概率 - α：平衡系数（通常 0.01~0.1）

直觉：当某个专家 f_i 和 P_i 都高时，乘积大，loss 大。优化会推动 token 更均匀地分配。

理想状态：每个专家处理 1/N 的 token。

精简版：

MoE 容易出现专家坍缩（大部分 token 涌向少数专家）。负载均衡 Loss = α・N・Σ(f_i・P_i) 惩罚不均匀分配，推动 token 均匀路由到各专家。

Q7: YaRN 长度外推的原理是什么？

详细版：

问题：RoPE 在训练长度内效果好，但超出训练长度后，位置编码的旋转角度超出了训练时见过的范围，注意力分数和 PPL 急剧恶化。

YaRN（Yet another RoPE extensioN）是一种动态调整 RoPE 以支持更长上下文的方法，它综合了以下技术：

1. **NTK-aware 插值**：不是简单地线性插值位置编码（压缩所有频率），而是根据频率的不同做差异化处理：
2. 高频分量（捕获局部关系）：不做或少做插值
3. 低频分量（捕获全局关系）：做更多的插值
4. **实现方式**：修改 RoPE 的 base 参数：`base_new = base × (scale_factor)^{dim/(dim-2)}` 这使得不同频率的缩放比例不同。
5. **注意力温度缩放**：对注意力分数做额外的缩放，补偿外推时的信息熵变化。

MiniMind 的配置： - rope_theta = 1e6（比默认的 10000 大 100 倍） - 更大的 base 本身就有利于长度外推 - 支持 YaRN 方案做进一步扩展

精简版：

YaRN 通过 NTK-aware 插值差异化处理不同频率的 RoPE：高频（局部关系）保持不变，低频（全局关系）做更多插值。MiniMind 的 rope_theta=1e6 本身有利于外推，并支持 YaRN 进一步扩展。

Q8: 文本生成策略有哪些？各有什么特点？

详细版：

策略	公式/方法	特点
Greedy	token = argmax(logits)	确定性、快、容易重复
Temperature	p = softmax(logits/T)	T<1 更确定，T>1 更随机
Top-K	只保留概率最高的 K 个 token	避免低概率 token
Top-P (Nucleus)	保留累积概率达到 P 的最小集合	自适应候选集大小
Beam Search	维护 B 条最优序列	常用于翻译，LLM 较少用
Repetition Penalty	对已生成的 token 降低概率	减少重复

常见组合：Temperature + Top-P（先调温度再截断）

各参数的影响： - Temperature = 0（≈Greedy）：最确定但最无聊 - Temperature = 0.7 + Top-P = 0.9：常用的"创意写作"参数 - Temperature = 0.1 + Top-P = 0.95：常用的"精确回答"参数 - Top-K = 50：固定候选集大小，不如 Top-P 灵活

Top-K vs Top-P： - Top-K 固定候选数量，但不同 token 位置需要的候选数不同 - Top-P 自适应：确定性强的位置（如 "Barack" 后面几乎一定是 "Obama"）候选少，不确定的位置候选多

精简版：

Greedy 取 argmax；Temperature 控制分布锐度；Top-K 保留前 K 个；Top-P 保留累积概率达 P 的集合（自适应）。常用组合：T=0.7 + Top-P=0.9。Top-P 比 Top-K 更灵活。

Q9: FlashAttention 的原理是什么？它为什么更快？

详细版：

核心问题：标准注意力需要在 GPU HBM（高带宽内存）中存储 n×n 的注意力矩阵，这个矩阵的读写是瓶颈。

FlashAttention 的核心思想：IO-aware，利用 GPU 的内存层次结构（SRAM 快但小，HBM 慢但大），通过 **分块计算（Tiling）** 避免在 HBM 中存储完整的注意力矩阵。

关键技术： 1. **分块（Tiling）**：将 Q、K、V 分成小块，每次只在 SRAM 中处理一小块 2. **在线 Softmax**：在不存储完整注意力矩阵的情况下，正确计算 softmax（使用 Online Softmax 算

法：维护 running max 和 running sum） 3. **反向传播优化**：不存储注意力矩阵，反向传播时重新计算（recomputation），用计算换内存

效果： - **速度**：比标准实现快 2-4 倍（减少 HBM 读写） - **内存**：O(n) 而非 $O(n^2)$（不存储注意力矩阵） - **数学上等价**：输出结果完全相同

注意：FlashAttention 没有改变注意力的计算复杂度（仍然 $O(n^2d)$），只是通过优化内存访问模式减少了实际运行时间。

精简版：

FlashAttention 将 QKV 分块在 SRAM 中计算，避免在 HBM 中存储 n^2的注意力矩阵。用 Online Softmax 实现分块正确的 softmax。速度快 2-4 倍，内存 $O(n^2)\rightarrow O(n)$，输出数学等价。

Q10: 模型量化是什么？INT8/INT4 量化有什么区别？

详细版：

量化将模型权重从高精度（FP32/FP16）压缩到低精度（INT8/INT4），减少模型大小和推理时的内存占用。

INT8 量化（8 bit）： - 将 FP16 权重映射到 [-128, 127] 的整数 - 模型大小减半（16bit → 8bit） - 质量损失很小 - 公式：w_int8 = round(w_fp16 / scale)，scale = max(|w|) / 127

INT4 量化（4 bit）： - 将权重映射到 4 bit 整数（16 个值） - 模型大小为 FP16 的 1/4 - 质量损失较明显，需要更精细的量化策略（如分组量化）

量化方法： - **PTQ（Post-Training Quantization）**：训练后直接量化，简单但可能损失较大 - **QAT（Quantization-Aware Training）**：训练时模拟量化效果，质量更好但需要重新训练 - **GPTQ**：基于二阶信息的 PTQ，目前效果最好的 PTQ 方法之一 - **AWQ**：保护重要权重通道不量化

精度	模型大小（per param）	质量	速度
FP16	2 bytes	最好	基准
INT8	1 byte	接近 FP16	更快（INT8 计算更快）
INT4	0.5 bytes	有损失	最快

精简版：

量化将 FP16 权重压缩为 INT8（模型减半）或 INT4（减为 1/4）。INT8 质量损失小，INT4 需要分组量化等技巧。方法有 PTQ（训练后量化）和 QAT（训练感知量化）。

Q11: vLLM 和 llama.cpp 有什么区别？各适合什么场景？

详细版：

特性	vLLM	llama.cpp
语言	Python + CUDA	C/C++
后端	GPU（CUDA）	CPU / GPU（GGML/GGUF）
核心优化	PagedAttention	量化（GGUF 格式）
适合场景	高并发 GPU 推理服务	边缘设备 / CPU 推理 / 轻量部署
吞吐量	高（PagedAttention 优化内存碎片）	中等
支持量化	FP16/INT8/AWQ/GPTQ	INT4/INT8/FP16 等多种 GGUF 格式
API	OpenAI 兼容 API	命令行 / API

vLLM 的 PagedAttention： - 借鉴操作系统的虚拟内存分页思想 - KV-Cache 不需要连续内存，按"页"分配 - 避免 KV-Cache 的内存碎片和预分配浪费 - 支持更大的 batch 和更高的并发

llama.cpp 的优势： - 纯 C/C++ 实现，无 Python/CUDA 依赖 - 可在 CPU 上运行，支持 macOS Metal、OpenCL 等 - 量化格式丰富，INT4 模型可以在消费级设备上运行

精简版：

vLLM 是 GPU 高并发推理引擎，核心是 PagedAttention 优化 KV-Cache 内存。llama.cpp 是 C++ 实现，支持 CPU/边缘设备，核心是 GGUF 格式量化。前者适合服务端，后者适合轻量部署。

Q12: Tool Calling（工具调用）是怎么实现的？

详细版：

Tool Calling 让 LLM 能够调用外部工具（如搜索引擎、计算器、API）来补充自身能力。

实现流程： 1. **定义工具**：用 JSON Schema 描述可用工具的名称、参数、功能 2. **SFT 训练**：用包含工具调用的对话数据训练模型，让模型学会生成结构化的工具调用指令 3. **推理流程**：

```
User: 今天北京天气怎么样？ Assistant: <tool_call>{"name": "weather", "arguments": {"city": "北京"}}</tool_call> System: [工具返回] {"temp": "22°C", "weather": "晴"}
```

`Assistant: 今天北京天气晴朗，气温 22° C。` 4. **格式约定**：模型输出特殊 token 或特定格式标记工具调用 5. **执行引擎**：外部程序解析模型输出、调用工具、将结果注入对话

训练数据构造： - 构造包含 tool_call 标记的对话数据 - 模型需要学会：何时调用工具、调用哪个工具、传什么参数 - SFT loss 仍然只计算 assistant 部分

精简版：

Tool Calling 通过 SFT 让模型学会在需要时生成结构化的工具调用指令（如 JSON 格式）。外部引擎解析指令、执行工具、将结果回注对话。训练时用包含工具调用的对话数据。

Q13: Adaptive Thinking（自适应思考）是什么概念？

详细版：

Adaptive Thinking 让模型根据问题的复杂度自适应地调整"思考深度"：

- **简单问题**（如 "1+1=?"）：直接回答，不需要长篇推理
- **复杂问题**（如 "证明勾股定理"）：展开详细的推理链

实现方式： 1. **思考 token**：引入特殊的 `<think>...</think>` 标记，模型在标记内做推理 2. **训练数据**：包含不同复杂度的问题，简单问题的 `<think>` 部分较短或为空，复杂问题有详细推理 3. **模型学会**：根据输入的复杂度动态决定思考链的长度

与 Chain-of-Thought 的关系： - CoT：始终输出推理过程 - Adaptive Thinking：根据需要决定是否输出、输出多少推理过程 - 更高效：简单问题不浪费 token

精简版：

Adaptive Thinking 让模型根据问题复杂度自适应调整推理深度。通过 `<think>` 标记实现，简单问题直接回答，复杂问题展开推理。比固定 CoT 更高效。

Q14: 模型格式转换有哪些常见场景？

详细版：

格式	说明	适用场景
PyTorch (.pt/.bin)	PyTorch 原生格式	训练、研究
Safetensors (.safetensors)	HuggingFace 安全格式	替代 pickle，更安全

格式	说明	适用场景
GGUF	llama.cpp 的量化格式	CPU/边缘设备推理
ONNX	开放神经网络交换格式	跨框架部署
TensorRT	NVIDIA 优化推理引擎格式	GPU 高性能推理

常见转换流程： 1. 训练完成 → PyTorch 格式 2. 上传 HuggingFace → 转换为 Safetensors 3. 本地部署 → 转换为 GGUF（llama.cpp 的 convert 工具） 4. GPU 生产部署 → 转换为 TensorRT

MiniMind 的格式转换： - 训练输出 PyTorch 格式 - 可转为 HuggingFace 格式（兼容 transformers 库） - 可转为 GGUF 格式在 llama.cpp 上运行

精简版：

常见格式：PyTorch（训练）→ Safetensors（HuggingFace）→ GGUF（llama.cpp 边缘部署）/ TensorRT（GPU 高性能部署）/ ONNX（跨框架）。

Q15: 推理延迟优化有哪些方法?

详细版：

方法	效果	描述
KV-Cache	必备	缓存已计算的 KV，避免重复计算
FlashAttention	2-4x 加速	IO-aware 分块计算，减少 HBM 访问
量化	INT8 加速 50%+	低精度计算，减少内存和带宽
Speculative Decoding	2-3x 加速	小模型草拟多个 token，大模型一次验证
Continuous Batching	吞吐提升 5-10x	请求级别的动态 batching
PagedAttention	内存利用率提升	消除 KV-Cache 内存碎片
Tensor Parallelism	延迟降低	将模型切分到多卡并行推理
Kernel Fusion	10-20% 加速	合并多个小算子减少 kernel launch 开销
GQA/MQA	KV-Cache 减少	减少 KV 头数，降低内存和带宽

按优先级排序： 1. KV-Cache（基础，不用就别做 LLM） 2. 量化（ROI 最高） 3. FlashAttention（几乎免费的加速） 4. Continuous Batching（服务端必备） 5. 其他（根据场景选择）

精简版：

优先级：KV-Cache（必备）→ 量化（ROI 最高）→ FlashAttention（免费加速）→ Continuous Batching（提吞吐）→ Speculative Decoding/TP 等。

Q16: Speculative Decoding（推测解码）的原理是什么？

详细版：

核心思想：用一个小而快的模型（Draft Model）先快速生成多个 token 的草稿，再用大模型（Target Model）一次性验证。

流程： 1. Draft Model 自回归生成 K 个 token：$t_1, t_2, ..., t_K$ 2. Target Model 接收 prefix + $[t_1, ..., t_K]$，一次前向传播得到每个位置的概率分布 3. 逐位验证：比较 Draft Model 和 Target Model 在每个位置的概率 4. 如果 Target 同意 Draft 的选择（概率足够高），接受该 token 5. 遇到第一个不接受的位置，从 Target Model 的分布中重新采样 6. 被接受的 token 都是精确的（与纯用 Target Model 生成的分布完全一致）

加速原理： - 大模型的主要开销在前向传播，但验证 K 个 token 和生成 1 个 token 的前向传播时间接近（同一 batch） - 如果 Draft Model 足够好（接受率高），一次验证可以确认 K 个 token，等效加速 K 倍

精简版：

用小模型快速草拟 K 个 token，大模型一次前向验证。被接受的 token 与大模型直接生成分布一致。验证 K 个 token 的成本接近生成 1 个，可加速 2-3 倍。

Q17: Continuous Batching 和 Static Batching 有什么区别？

详细版：

Static Batching： - 一批请求一起进入，一起结束 - 如果某个请求很早生成完毕（如 EOS），它的 GPU 资源闲置直到整批完成 - 浪费严重

Continuous Batching（也叫 Iteration-level Batching）： - 每生成一个 token 后，检查是否有请求完成（遇到 EOS） - 完成的请求立即移出，新请求立即加入 - GPU 资源始终被充分利用

效果： - 吞吐量提升 5-10 倍 - 延迟更稳定 - GPU 利用率显著提高

vLLM 和 TGI（Text Generation Inference）都实现了 Continuous Batching。

精简版：

Static Batching 等最慢的请求完成才能处理新请求，GPU 浪费严重。Continuous Batching 每步动态增删请求，GPU 利用率高，吞吐提升 5-10 倍。

Q18: Prefill 和 Decode 阶段有什么区别?

详细版：

LLM 推理分为两个阶段：

Prefill（预填充）阶段： - 处理整个 prompt（输入序列） - 一次性并行计算所有 prompt token 的 KV-Cache - 计算密集型（compute-bound），GPU 计算利用率高 - 类似训练的前向传播

Decode（解码）阶段： - 逐 token 自回归生成 - 每步只处理一个新 token - 内存密集型（memory-bound），大量时间花在读取 KV-Cache 上 - GPU 计算利用率低（矩阵太小无法充分利用并行度）

优化方向不同： - Prefill：FlashAttention、Tensor Parallelism - Decode：KV-Cache 优化、量化减少带宽、Speculative Decoding

Chunked Prefill：将长 prompt 分块处理，与 decode 请求交错调度，提高整体效率。

精简版：

Prefill 并行处理整个 prompt 生成 KV-Cache（计算密集），Decode 逐 token 生成（内存密集）。优化方向不同：Prefill 用 FlashAttention，Decode 用 KV-Cache 优化和量化。

Q19: 什么是 Tensor Parallelism 和 Pipeline Parallelism?

详细版：

Tensor Parallelism（TP，张量并行）： - 将单个层的权重矩阵切分到多张 GPU 上 - 例如：将 FFN 的 W_1 按列切分到 2 张卡，各计算一半，再合并 - 需要每层通信（AllReduce），适合同一节点内的多卡 - **降低单请求延迟**

Pipeline Parallelism（PP，流水线并行）： - 将不同层分配到不同 GPU - 例如：8 层模型，GPU0 处理 1-4 层，GPU1 处理 5-8 层 - 层间传递激活值，通信量小 - 存在流水线气泡（bubble），需要微批次调度来减少

Data Parallelism（DP，数据并行）： - 每张 GPU 持有完整模型，处理不同数据 - DDP 的方式

组合使用（3D Parallelism）： - 大规模训练通常同时使用 DP + TP + PP - 例如：4 节点各 8 卡 = 32 卡，TP=8（节点内），PP=2（节点间），DP=2

精简版：

TP 切分单层权重到多卡（降延迟，通信频繁）；PP 不同层放不同卡（通信少但有气泡）；DP 每卡完整模型处理不同数据。大规模训练三者组合使用。

Q20: BF16 和 FP16 有什么区别?

详细版：

特性	FP16	BF16
总位数	16	16
指数位	5	8
尾数位	10	7
数值范围	±65504	$\pm 3.4 \times 10^{38}$（与 FP32 相同）
精度	约 3-4 位有效数字	约 2-3 位有效数字
溢出风险	高（范围小）	低（范围与 FP32 相同）

BF16 的优势： - 动态范围与 FP32 相同，几乎不会溢出 - 混合精度训练更稳定，不太需要 Loss Scaling - A100/H100 等新 GPU 对 BF16 有原生支持

FP16 的优势： - 精度更高（尾数多 3 位） - 在不溢出的情况下数值更准确 - 更多 GPU 支持（包括较老的 GPU）

选择建议： - 如果 GPU 支持 BF16（A100+），优先使用 BF16 - 3090 支持 BF16，但 Tensor Core 对 BF16 的加速不如 A100 - FP16 + Loss Scaling 在大多数场景也能工作

精简版：

BF16 指数 8 位（范围大不易溢出）、尾数 7 位（精度低）；FP16 指数 5 位（范围小易溢出）、尾数 10 位（精度高）。BF16 训练更稳定，A100+ 优先用 BF16。

Q21: 如何评估 LLM 的生成质量?

详细版：

自动评估指标： 1. **Perplexity (PPL)**：语言模型的基础指标，越低越好 2. **BLEU/ROUGE**：与参考文本的重叠度，适合翻译/摘要 3. **Pass@K**：代码生成的通过率（如 HumanEval） 4. **Accuracy**：在选择题 benchmark 上的准确率（MMLU, HellaSwag 等）

人工评估： - 流畅性、信息准确性、有帮助程度 - A/B 对比测试 - Elo 排名（如 Chatbot Arena）

LLM-as-Judge： - 用强 LLM（如 GPT-4）评估弱 LLM 的输出 - 成本低于人工评估，但有偏见

MiniMind 的评估： - PPL 作为预训练质量指标 - 对话质量主要靠人工评估（小模型在 benchmark 上分数不高）

精简版：

自动指标：PPL（语言模型质量）、BLEU/ROUGE（文本匹配）、Accuracy（benchmark）。人工评估：流畅性、准确性。LLM-as-Judge 用强模型评弱模型。

Q22: 什么是 GradCheckpointing（梯度检查点/重计算）？

详细版：

问题：训练时需要保存所有中间激活值（activations）用于反向传播，显存占用很大。

GradCheckpointing 的做法： - 前向传播时不保存某些层的中间激活值 - 反向传播到这些层时，重新执行前向传播计算中间值 - 用**时间换空间**

效果： - 显存占用大幅减少（通常减少 60-70%） - 训练速度降低 20-30%（因为要重新计算） - 可以训练更大的模型或使用更大的 batch_size

PyTorch 实现：

```
from torch.utils.checkpoint import checkpoint

class TransformerBlock(nn.Module):
    def forward(self, x):
        x = x + checkpoint(self.attention, self.norm1(x))
        x = x + checkpoint(self.ffn, self.norm2(x))
        return x
```

精简版：

GradCheckpointing：前向不保存中间激活值，反向时重新计算。用时间（慢 20-30%）换空间（省 60-70% 显存），可训练更大模型或用更大 batch。